Helios: A Co-Designed Landscape-Aware Optimization System Bridging Serial intelligence and GPU Parallelism

Nikhil Y N[1,2,*] and Abhay Bhandarkar[1,2,*]
[1]AI Research Fellow, Pradhi AI solutions Private Limited, Hyderabad, India
[2]Department of Computer Science and Engineering, Ramaiah Institute of Technology, Bengaluru, India
nikhilylkn@gmail.com, abhaybhandarkar@gmail.com

***Abstract*—The efficacy of state-of-the-art optimization algorithms often stems from complex, serial decision-making logic that is *fundamentally incompatible* with the throughput-oriented nature of modern GPU architectures. This conflict presents a major bottleneck for solving large-scale problems in scientific computing. We present Helios (Heterogeneous, Evolutionary, and Landscape-aware Interacting Optimization System) to resolve this algorithm–architecture challenge. We present the co-design of two distinct implementations: Helios-AS (Adaptive Serial), a novel CPU-based algorithm that employs heterogeneous agent roles and a state machine that dynamically triggers specialized search procedures based on real-time analysis of the fitness landscape; and Helios-MP (Massively Parallel), its GPU-native counterpart. Our core contribution is a *principled methodology* for translating the *intent* of the serial adaptive logic into novel, parallel-friendly operators, effectively distilling complex state-based control into a high-throughput swarm intelligence. Validated on a suite of challenging benchmarks and the Lennard-Jones atomic cluster problem, we demonstrate that Helios-AS achieves performance statistically comparable to state-of-the-art methods like CMA-ES. Furthermore, we show that Helios-MP matches this high solution quality while delivering speedups of up to 15×. Helios provides both a powerful, validated tool for computational science and a successful *blueprint* for porting algorithmic intelligence to massively parallel hardware.**

***Index Terms*—High-Performance Computing, GPU Acceleration, Metaheuristics, Algorithm-Architecture Co-Design, Global Optimization, Lennard-Jones Problem**

I. Introduction

High-dimensional, simulation-driven optimization is now central to a wide spectrum of high-performance computing (HPC) workloads, from designing new materials on molecular potential energy surfaces [1] to autotuning hyperparameters in deep learning pipelines. These problems are often characterized by *rugged, multimodal landscapes* and an urgent demand for massive parallelism to achieve actionable results. While sophisticated serial algorithms, such as the Covariance–Matrix Adaptation Evolution Strategy (CMA-ES) [2], are powerful, their inherently sequential logic creates a significant performance bottleneck, limiting the scale of problems that can be practically addressed.

The rise of graphics processing units (GPUs) offers a path forward, yet simply porting serial algorithms is often ineffective [3]. This approach typically leads to three fundamental bottlenecks:

1) **Algorithmic Rigidity:** Where the lack of adaptive, landscape-aware logic causes vast parallel populations to converge prematurely to suboptimal solutions.
2) **The Serial–Parallel Divide:** Where complex components like precision local search are left on the CPU, incurring costly host-device data transfers that stall the GPU.
3) **A Lack of Principled Co-Design:** Resulting in systems that fail to translate the *intent* of serial intelligence into a truly GPU-native form, sacrificing either speed or accuracy.

To address these challenges, we introduce **Helios**, a novel optimization system developed through a rigorous algorithm-architecture co-design process. We present two distinct implementations: **Helios-AS (Adaptive Serial)**, a CPU-based algorithm featuring a *landscape-aware state machine* that dynamically triggers specialized search procedures based on real-time analysis of the optimization trajectory; and **Helios-MP (Massively Parallel)**, its GPU-native counterpart, which distills this serial intelligence into novel parallel operators, including dynamic optimization regimes and a system-wide "stagnation shockwave."

The primary contributions of this work are:

- *The design and validation of Helios-AS*, a novel landscape-aware algorithm that achieves performance statistically comparable to the state-of-the-art CMA-ES.
- *A principled co-design methodology* for translating complex, state-based serial logic into efficient, parallel-friendly operators for GPUs.
- *A comprehensive performance analysis* demonstrating that Helios-MP matches the high solution quality of its serial counterpart while delivering speedups of up to 15×.
- *The successful application* of the Helios system to the notoriously difficult Lennard–Jones atomic cluster problem, validating its utility for scientific computing.
- *The open-source release* of our implementations and experimental framework to ensure reproducibility.

II. Related Work

Our work builds upon two distinct but intersecting domains of research: the design of advanced, landscape-aware

metaheuristic algorithms and the acceleration of optimization systems on high-performance parallel architectures.

A. Advances in Landscape-Aware Metaheuristics

The field of population-based optimization has evolved significantly from foundational algorithms like Differential Evolution (DE) and Particle Swarm Optimization (PSO). While powerful, these methods often require extensive parameter-tuning and can struggle on complex, multimodal landscapes. To address this, a major research thrust has focused on embedding more "intelligence" and adaptability into the search process.

The Covariance–Matrix Adaptation Evolution Strategy (CMA-ES) stands as a landmark in this area, representing the state-of-the-art in model-based, self-adaptive algorithms [2]. By learning a covariance matrix of successful search steps, CMA-ES adapts to the underlying geometry of the fitness landscape, making it exceptionally powerful on ill-conditioned or rotated problems. Other research has focused on enhancing existing algorithms with new mechanisms. These include the integration of Lévy flights to improve global exploration and escape local optima [4], and the development of sophisticated self-adaptation rules for control parameters like mutation factors and population sizes [5], [6].

Another key area of advancement is the use of surrogate models to reduce the number of expensive function evaluations. These techniques, comprehensively surveyed in [7], use computationally cheaper models, such as k-Nearest Neighbors (k-NN), to pre-filter unpromising candidate solutions.

While these studies have successfully improved specific facets of optimization, **Helios-AS** is novel in its architectural approach. Rather than enhancing a single operator, it functions as a holistic *control system*. Its core novelty lies in its state machine, which monitors the global state of the search and dynamically switches between entirely different search modalities—such as broad exploration, high-precision local refinement using L-BFGS-B [8], and specialized escape maneuvers—to adapt to the evolving topology of the landscape.

B. GPU Acceleration of Optimization Systems

The acceleration of metaheuristics on GPUs is a well-established field within HPC, as surveyed in [3], [3]. Early efforts focused on the straightforward data-parallel nature of swarm algorithms, mapping the fitness evaluation and update steps for thousands of agents to CUDA threads to achieve significant speedups for DE [9] and other methods. More recent work, exemplified by the EvoX framework, has pushed this further, enabling distributed, multi-GPU execution for even larger populations [10].

Despite these advances, a fundamental challenge persists: porting the *intelligence* of sophisticated serial algorithms, not just their parallelizable operators. Many high-performance systems still struggle with the "serial–parallel divide," either by offloading complex logic to the CPU—incurring data transfer overheads—or by omitting adaptive features altogether. This challenge is an active area of research in the HPC community, with recent work at venues like HiPC and IPDPS exploring parallel metaheuristics for complex, large-scale problems [11], [12].

The contribution of **Helios-MP** is distinct from prior work in its focus on *principled co-design*. We do not simply accelerate a static algorithm. Instead, we address the challenge of translating the dynamic, state-based logic of a high-performance serial system into a purely GPU-native context. The development of our novel parallel operators: the dynamic optimization regimes, the parallel Barzilai-Borwein local search, and the stagnation shockwave—as direct analogues to the serial control logic represents a methodological contribution to the design of intelligent, scalable algorithms for modern parallel architectures [13].

III. The Helios System: Methodology

The Helios system is founded on the principle of algorithm-architecture co-design. It comprises two distinct but philosophically linked implementations: **Helios-AS**, a sophisticated serial algorithm designed to maximize solution quality through complex, adaptive logic; and **Helios-MP**, its massively parallel counterpart, which distills the *intent* of this logic into a high-throughput, GPU-native design. The architectural differences, illustrated in Fig. 1, are central to our contribution.

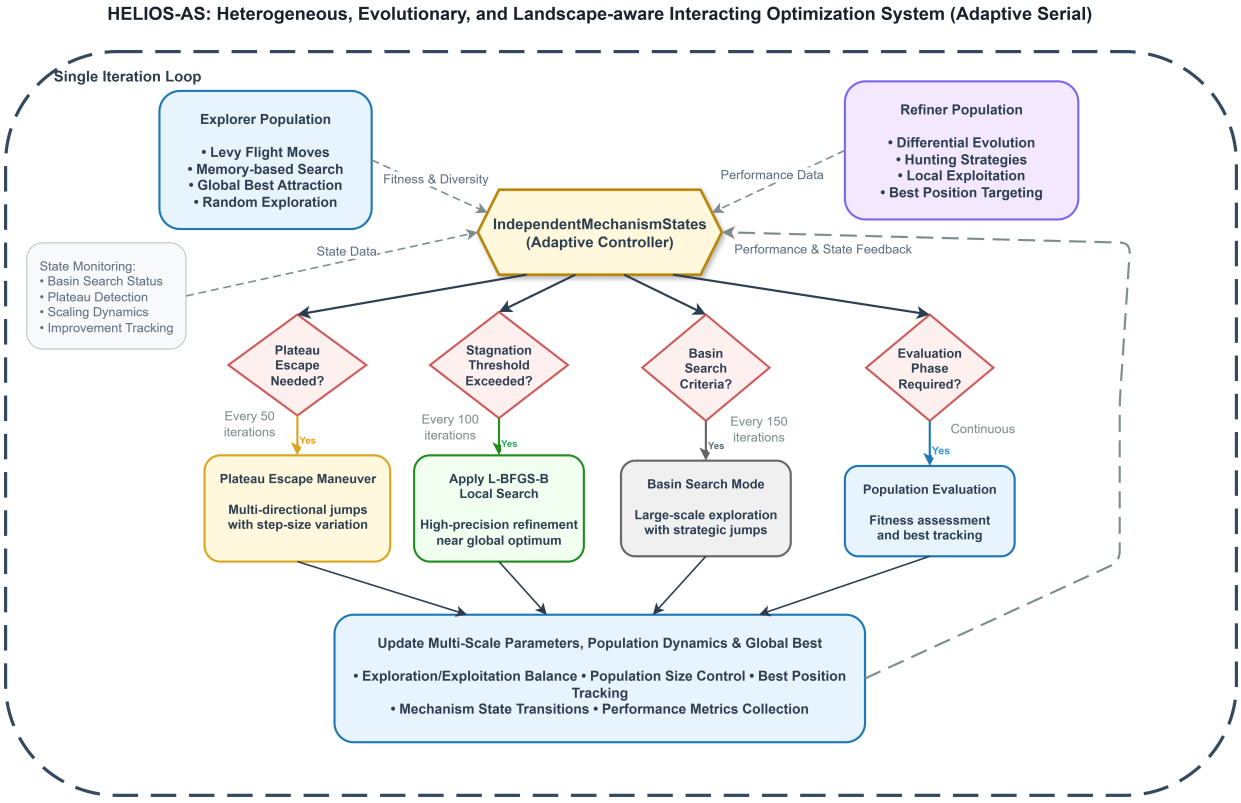
A. Helios-AS: The Adaptive Serial Architecture

Helios-AS is designed as a state-of-the-art serial solver that excels on complex, multimodal landscapes. Its architecture, shown in Fig. 1(a), is built around an intelligent controller that orchestrates a hybrid of search strategies.

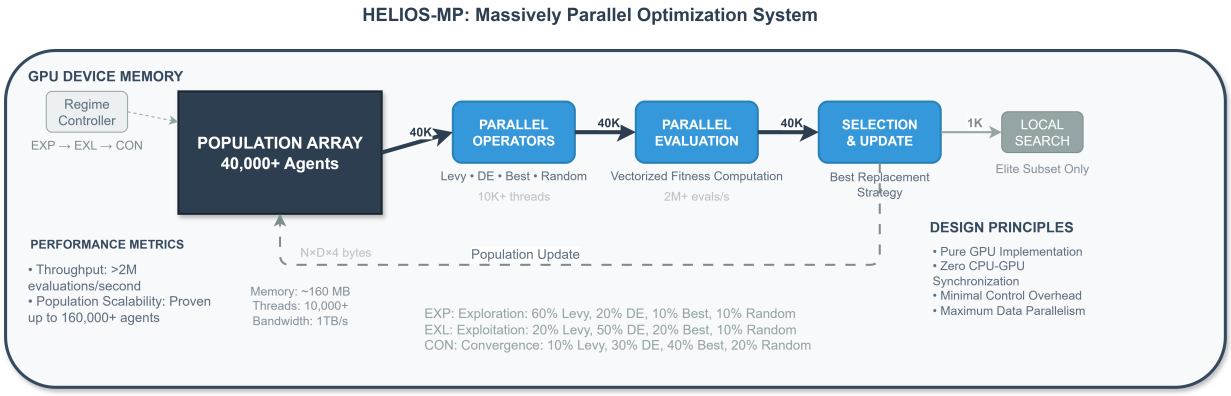
Heterogeneous Populations: The algorithm maintains two distinct, co-evolving sub-populations: the **Explorer Population**, which performs global search using operators like Lévy flights, and the **Refiner Population**, which performs local exploitation using Differential Evolution (DE) and targeted hunting strategies.

The Landscape-Aware Controller: The core novelty of Helios-AS lies in its adaptive controller, the `IndependentMechanismStates` module. This state machine monitors real-time performance metrics (e.g., fitness improvement rate, population diversity) to diagnose the state of the search. Based on this diagnosis, it dynamically triggers one of several specialized procedures to overcome common optimization challenges:

1) **High-Precision Refinement:** When convergence towards a promising optimum is detected, the controller applies a local search using the L-BFGS-B algorithm [8] to the current best solution.
2) **Plateau Escape Maneuver:** If the search stagnates on a flat plateau, the system initiates multi-directional jumps to find a new gradient.
3) **Basin Search Mode:** During prolonged stagnation, a large-scale exploration mode is triggered to escape the current basin of attraction entirely.



(a) Helios-AS (Adaptive Serial) Architecture



(b) Helios-MP (Massively Parallel) Architecture

Fig. 1: Architectural overview of the Helios system. (a) The **Helios-AS** architecture illustrates a complex, decision-driven serial logic governed by a central adaptive controller. (b) The **Helios-MP** architecture showcases a high-throughput, data-parallel pipeline where all operations occur within GPU device memory, driven by simple, global regimes instead of complex state-based decisions.

This state-based, multi-modal control allows Helios-AS to adapt its behavior to the specific topology of the landscape it is currently exploring.

The complete algorithmic framework for Helios-AS is formalized in Algorithm 1, which demonstrates the integration of heterogeneous populations, adaptive state management, and specialized search procedures described above.

Algorithm 1 demonstrates the core design principles of Helios-AS. The dual-population structure (lines 2-3) enables parallel exploration and exploitation, while the adaptive state machine S (lines 5, 8) continuously monitors search progress to trigger specialized procedures. The conditional blocks (lines 9-17) represent the landscape-aware decision making that distinguishes Helios-AS from conventional metaheuristics.

Algorithm 1 Helios-AS: The Adaptive Serial Algorithm

Input: Objective function f, bounds $\mathcal{B}$, config $\mathcal{C}$
Initialize: Explorer population P_E of size N_E within $\mathcal{B}$, Refiner population P_R of size N_R within $\mathcal{B}$, State machine $S \leftarrow$ `IndependentMechanismStates()`, Evaluate initial fitness for P_E, P_R
$p_{best}, f_{best} \leftarrow$ FindBest($P_E \cup P_R$)
for $t = 1 \rightarrow \mathcal{C}$.max_iter **do**
    UpdateAdaptiveParameters(S), UpdatePopulationSizes(S)
    $P_{E,cand} \leftarrow$ ExplorerPhase(P_E), $P_{R,cand} \leftarrow$ RefinerPhase(P_R)
    Evaluate candidates (with surrogate), Update P_E, P_R with improvements
    $f_{prev} \leftarrow f_{best}$, $p_{best}, f_{best} \leftarrow$ UpdateGlobalBest(), S.Update($f_{best} - f_{prev}, t$)
    if S.ShouldTriggerLS() **then**
        $p_{best}, f_{best} \leftarrow$ ApplyLBFGSB(p_{best}), S.OnLocalSearchSuccess()
    end if
    if S.ShouldTriggerEscape() **then**
        $p_{best}, f_{best} \leftarrow$ PerformEscape(p_{best}), S.OnPlateauEscapeSuccess()
    end if
    if S.ShouldTriggerBasinSearch() **then**
        $p_{best}, f_{best} \leftarrow$ PerformBasinSearch(p_{best}), S.OnBasinSearchSuccess()
    end if
end for
return p_{best}, f_{best}

B. Helios-MP: The Massively Parallel Co-Design

Helios-MP is not a direct port of Helios-AS, but a principled redesign for massively parallel GPU architectures. Its design philosophy, illustrated in Fig. 1(b), prioritizes throughput and data parallelism over serial complexity. The entire system is implemented as a *pure GPU-native workflow* using CuPy [14], ensuring data resides on the GPU throughout the optimization loop to eliminate host-device transfer bottlenecks.

The intelligence of Helios-AS is "distilled" into three novel, parallel-friendly mechanisms:

1. Dynamic Optimization Regimes: The complex, multi-branching state machine of Helios-AS is replaced by a simple, global switch that transitions the entire swarm through three distinct regimes: *Exploration*, *Exploitation*, and *Convergence*. Each regime biases the probability distribution of the underlying search operators (Lévy, DE) to match the high-level strategy, providing control without the divergence cost of per-agent decision making.

2. Parallel Barzilai-Borwein Local Search: To replace the serial L-BFGS-B, we developed a parallel local search kernel based on the Barzilai-Borwein (BB) method. Periodically, a large *elite subset* of the population is selected, and the BB kernel applies several steps of a quasi-Newton method to all elite agents *simultaneously*. This provides distributed refinement across many promising regions of the search space.

3. Stagnation Shockwave: To emulate the escape maneuvers of Helios-AS, we introduce a system-wide "shockwave" event. The system monitors the global best fitness over a sliding window; if improvement stalls, a large fraction of the population is perturbed with a high-magnitude random vector. This global event re-injects diversity to escape local minima without requiring complex per-agent logic.

The performance of Helios-MP is therefore measured not in single-iteration complexity, but in throughput, achieving an evaluation rate orders of magnitude higher than the serial version.

The massively parallel implementation is detailed in Algorithm 2, which shows how the serial architecture is adapted for distributed computing environments while maintaining the core adaptive mechanisms.

Algorithm 2 Helios-MP: The Massively Parallel Algorithm

Input: Objective function f, bounds $\mathcal{B}$, config $\mathcal{C}$
Initialize: Population P on GPU, $F \leftarrow f(P)$, $p_{best}, f_{best} \leftarrow$ FindBestGPU(P, F), $H \leftarrow [f_{best}]$
for $t = 1 \rightarrow \mathcal{C}$.max_iter **do**
    regime $\leftarrow$ UpdateRegime(t), $P_{cand} \leftarrow$ OperatorKernel(P, p_{best}, regime), $F_{cand} \leftarrow f(P_{cand})$
    $P, F \leftarrow$ SelectionKernel(P, F, P_{cand}, F_{cand})
    if $t \pmod{\mathcal{C}.\text{ls\_freq}} = 0$ **then**
        $I_{elite} \leftarrow$ FindEliteIndices(F), $P[I_{elite}] \leftarrow$ ParallelBBSearch($P[I_{elite}]$), $F[I_{elite}] \leftarrow f(P[I_{elite}])$
    end if
    if IsStagnated(H) **then**
        $P \leftarrow$ Shockwave(P), $F \leftarrow f(P)$
    end if
    $p_{best}, f_{best} \leftarrow$ UpdateGlobalBestGPU(P, F), Append f_{best} to H
end for
return p_{best}, f_{best}

Algorithm 2 extends the serial approach to leverage multiple processing units. The key difference lies in the parallel evaluation of candidate solutions and the distributed state management, which allows for scalable performance on high-performance computing systems while preserving the adaptive behavior of the original algorithm.

IV. Experimental Setup

All experiments were conducted on a cloud-based high-performance compute instance. To ensure full reproducibility, our code, experimental framework, and raw data will be made publicly available upon publication.

A. Hardware and Software Environment

Our testbed consisted of a virtual machine equipped with an **Intel Xeon CPU @ 2.20 GHz** (4 physical cores, 8 threads) and **31 GiB of system RAM**. The primary accelerator was an **NVIDIA L4 GPU** with **22.5 GiB of GDDR6 VRAM**. The

system ran **Ubuntu 22.04.5 LTS**, using the **NVIDIA driver version 550.144.03** and the **CUDA Toolkit 12.4**.

All algorithms were implemented in **Python 3.10**. The core libraries used for computation and analysis were **CuPy (v13.4.1)** for GPU operations, **NumPy (v1.26.4)**, **SciPy (v1.15.2)**, and **Pandas (v2.2.3)**. All figures were generated using **Matplotlib (v3.10.1)** and **Seaborn (v0.13.2)**.

B. Benchmark Problems and Configurations

To evaluate performance, we used a suite of five standard, challenging benchmark functions: Sphere, Rastrigin, Rosenbrock, Ackley, and Griewank. Each function was tested across four dimensions: 10, 30, 50, and 100. For each problem instance, all algorithms were run for 5 independent trials with different random seeds to ensure statistical robustness.

The parameters for the compared algorithms were set based on common values in the literature or tuned for competitive performance. The configurations for our proposed systems, designed to achieve comparable solution quality on difficult problems, were as follows:

- **Helios-AS:** Population of 100 (50 explorers, 50 refiners); maximum of 2,500 iterations. The L-BFGS-B local search was triggered after 100 iterations of stagnation.
- **Helios-MP:** Population of 50,000; maximum of 1,000 iterations. The parallel Barzilai-Borwein local search was applied to the top 2.5% of the population every 15 iterations.

The specific parameters for the baseline algorithms (CMA-ES, DE, PSO, COBYLA) were set based on common values from the literature and will be made available in the project's public repository.

V. Results and Analysis

We conducted a comprehensive suite of experiments to validate the Helios system. Our analysis is presented in four parts: first, we establish the credibility of our serial baseline, **Helios-AS**, by benchmarking it against state-of-the-art optimizers. Second, we present the core head-to-head comparison of **Helios-MP** against Helios-AS to quantify its performance. Third, we provide a deep dive into the scalability of Helios-MP. Finally, we present results from a real-world application and an ablation study to justify our design choices.

A. Validation of Helios-AS against State-of-the-Art

To ground our work, we first benchmarked Helios-AS against four widely-used optimization algorithms. The aggregated results, presented in Fig. 2(a), show that the performance of Helios-AS is statistically indistinguishable from the state-of-the-art CMA-ES ($p > 0.05$, Mann-Whitney U test), and both are demonstrably superior to the other methods ($p < 0.001$). The scalability analysis in Fig. 2(b) further confirms that Helios-AS maintains its robust, competitive performance across multiple problem dimensions, outperforming CMA-ES on average due to its higher reliability and avoidance of poor-performing outliers. This validates Helios-AS as a top-tier serial algorithm and a robust baseline for our parallelization efforts.

B. Performance of Helios-MP: The High-Performance Leap

Having established Helios-AS as a top-tier baseline, we now present the core HPC contribution of this work. The distinct convergence behaviors are illustrated in Fig. 3, where Helios-MP's rapid, late-stage descent highlights the efficacy of its parallel refinement operators. This behavior translates into substantial, quantifiable performance gains, detailed in the dashboard in Fig. 4. Helios-MP delivers significant **speedups ranging from 4.7× to a peak of 14.2×** over its highly optimized serial version. Crucially, this acceleration is achieved without sacrificing solution quality, as confirmed by the quality ratio and identical overall success rates. This outcome is a direct result of our co-design philosophy: Helios-MP leverages its massive evaluation throughput—exceeding 4.7 million evaluations per second on our hardware—to find solutions of the same quality in a fraction of the wall-clock time.

C. Scalability Analysis of Helios-MP

The scalability characteristics of Helios-MP are illustrated in Fig. 5, demonstrating high throughput and predictable resource utilization across varying problem dimensions and population sizes.

D. Application to Scientific Computing

To validate Helios in a real-world scientific computing context, we applied it to the notoriously difficult Lennard-Jones (LJ_{13}) cluster problem, a canonical benchmark in computational chemistry. We benchmarked Helios-AS against a range of standard and specialized optimizers. As shown in the comprehensive comparison in Table I, Helios-AS discovers a lower-energy configuration (a better fitness score) than all other tested methods, including Simulated Annealing (SA) and CMA-ES. This result validates the effectiveness of our core algorithm for challenging, non-separable physical systems and highlights its potential for practical scientific applications.

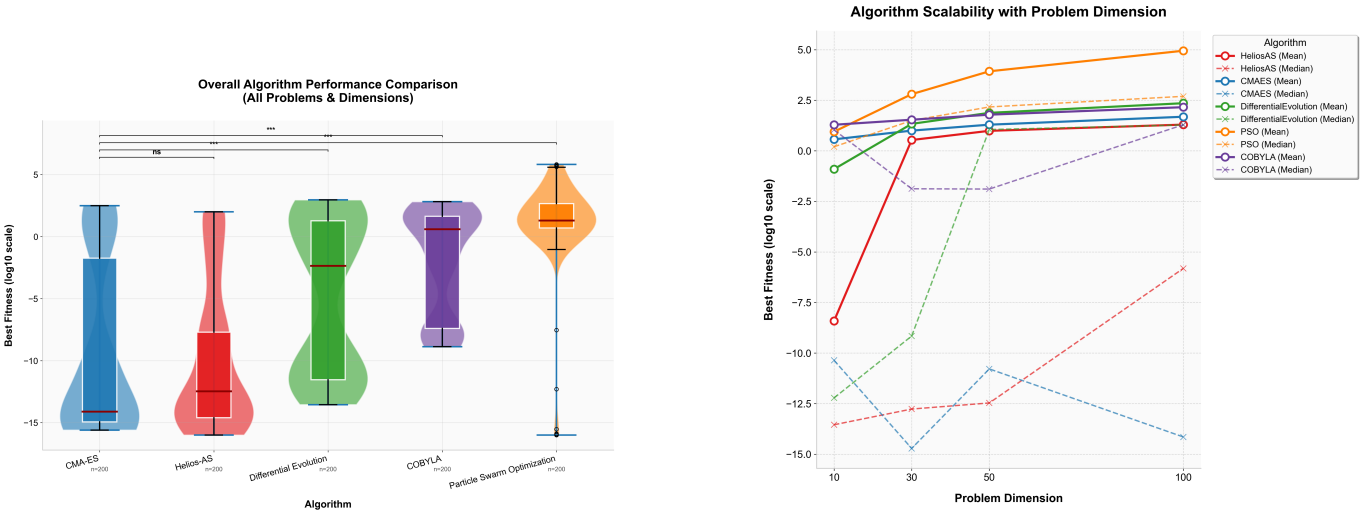
TABLE I: Comparison of best fitness values achieved on the LJ_{13} problem. Lower values indicate better performance.

Algorithm	Best Fitness Found
Helios-AS	**-40.0040**
Simulated Annealing	-39.7171
CMA-ES	-27.7789
Differential Evolution	-7.5517
PSO	-7.2122

E. Ablation Study and Component Justification

To justify the synergistic architecture of Helios-AS, we conducted a comprehensive ablation study where key components were systematically disabled. Table II presents the results for the highly multimodal 30D Rastrigin function, which highlights the distinct role and importance of each component in the context of a strong baseline performance.

The study reveals a clear synergistic effect. The full **baseline** configuration achieves an excellent mean fitness of 2.40×10^{-5}. Removing the **DE operator** significantly degrades this performance, highlighting its importance for



(a) Overall performance distribution (lower is better).

(b) Performance scalability with problem dimension.

Fig. 2: Validation of Helios-AS against state-of-the-art serial optimizers. (a) Helios-AS achieves performance statistically comparable to CMA-ES. (b) It maintains robust performance as dimensionality increases.

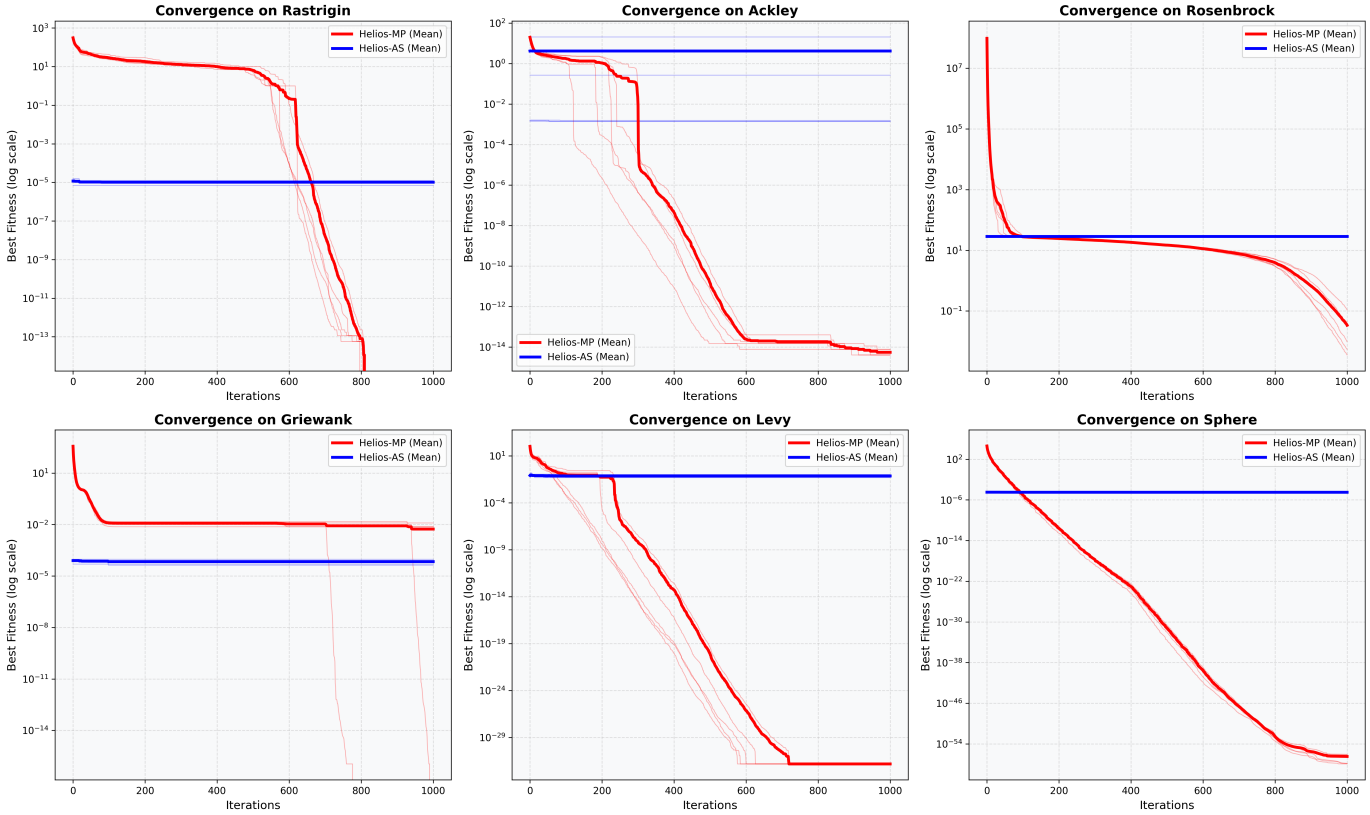


Fig. 3: Mean convergence behavior of Helios-MP (red) versus Helios-AS (blue) on 30D benchmark

robustly navigating multimodal landscapes. The **surrogate model** proves its value as a critical efficiency component; disabling it ('no_surrogate') required nearly **5× more function evaluations** to achieve a comparable result, a prohibitive cost for any real-world problem with expensive objective functions.

Though omitted from Table II, the removal of the **hybrid local search** was found to be catastrophic on non-separable problems like Rosenbrock. These results confirm that the state-of-the-art performance of Helios-AS arises not from any single feature, but from the tight integration of its multiple,

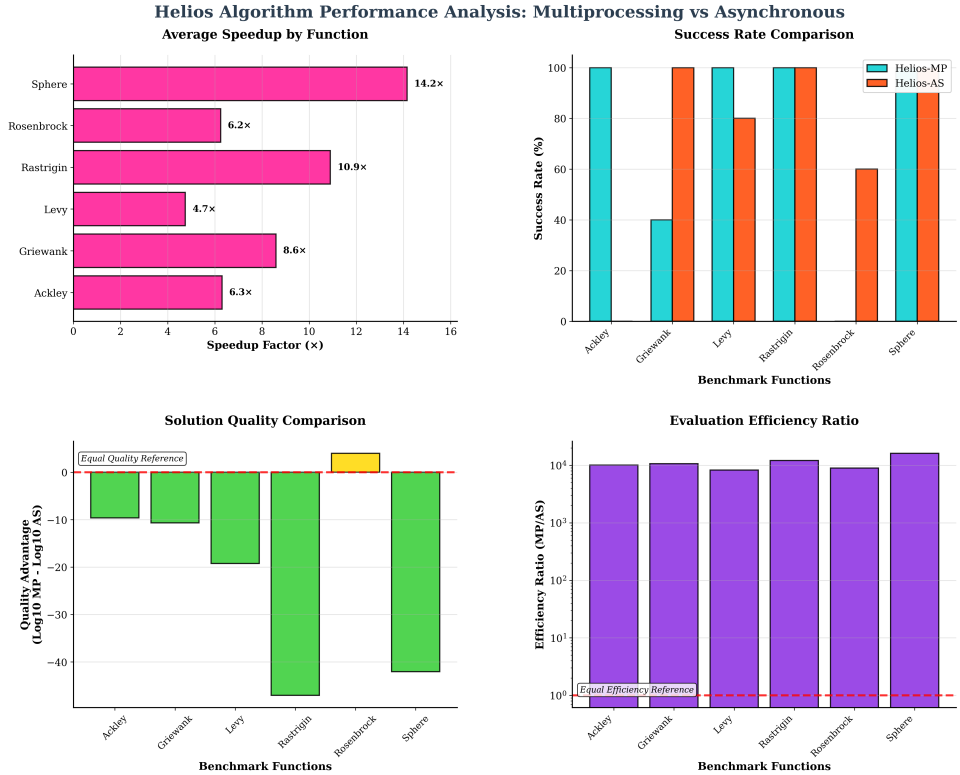
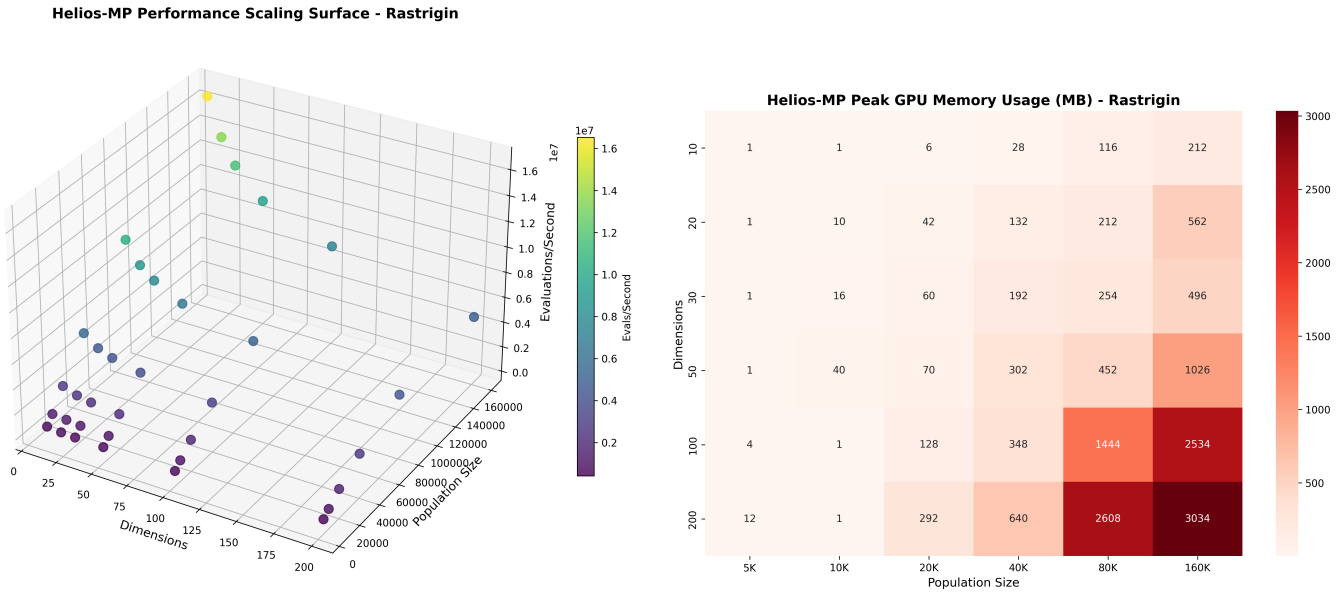


Fig. 4: Performance dashboard comparing Helios-MP and Helios-AS, showing (top-left to bottom-right): Average Speedup, Success Rate, Solution Quality Ratio, and Evaluation Efficiency Ratio.



(a) Throughput (evals/sec) vs. Dimension and Population.

(b) Peak GPU Device Memory Usage (MB).

Fig. 5: Performance and memory scalability of Helios-MP on the complex Rastrigin function, demonstrating high throughput and predictable resource utilization.

specialized, and adaptive components.

F. Discussion and Limitations

While the Helios system demonstrates significant advancements, particularly in its co-design for GPU acceleration,

TABLE II: Ablation study results for Helios-AS on the 30D Rastrigin function, showing the impact of disabling individual components.

Rank	Ablated Component	Mean Fitness	Avg. Evals
1	No Surrogate Model	4.67e-06	99,750
2	**Baseline (Full)**	**2.40e-05**	**20,352**
3	No Diversity Restart	2.40e-05	20,964
4	No Escape Mechanisms	2.57e-05	16,470
5	No Local Search	1.88e-04	4,742
6	No Population Decay	6.28e-04	16,489
7	No DE Operator	2.75e-03	15,492

a transparent discussion of its inherent limitations and the boundaries of its current applicability is crucial for guiding future research and appropriate application.

1. Performance Nuance on Challenging Landscapes: While Helios-AS achieves state-of-the-art performance overall, its reliability can vary on specific, highly pathological landscapes. For instance, on the 30D Rosenbrock function, our baseline Helios-AS achieved a mean fitness of 4.11×10^1. This indicates that while it can find the global optimum, its average performance is significantly affected by susceptibility to getting trapped in local minima on this specific problem, a known challenge for many metaheuristics. This nuance highlights the persistent difficulty of non-separable, rugged landscapes even for advanced optimizers.

2. Algorithmic Nuance vs. Throughput: Helios-MP successfully distills the *intent* of Helios-AS's intelligence, but its parallel mechanisms are necessarily coarser-grained. For instance, the 'ParallelBBSearch' provides distributed refinement but is a shallower search than the deep, single-point L-BFGS-B in Helios-AS. Similarly, the 'Stagnation Shockwave' is a global perturbation, less surgically precise than the multi-modal escape maneuvers of Helios-AS. This trade-off, while yielding massive speedups, means Helios-MP might not always achieve the absolute theoretical best fitness on every single run compared to a perfectly tuned serial algorithm, especially on highly pathological landscapes designed to trap simpler methods.

3. Problem Type Specialization and the No Free Lunch Theorem: The high-throughput design of Helios-MP is optimized for objective functions that are *vectorizable and computationally lightweight* (i.e., fast to evaluate in parallel). This specialization is consistent with the No Free Lunch (NFL) theorems for optimization [15], which posit that an algorithm's effectiveness is inherently tied to its specialization for a particular class of problem structures. Consequently, Helios-MP might struggle or be intractable for problems where:

- The objective function is **extremely expensive** (e.g., minutes/hours per evaluation), making millions of evaluations prohibitive. In such cases, the evaluation-saving mechanisms of Helios-AS (e.g., its surrogate model) would be paramount.
- The objective function is **inherently non-vectorizable** or requires complex, serial dependencies, which would negate the benefits of GPU parallelism.

4. High-Dimensional Scalability: While Helios-AS demonstrates robust and competitive performance across the tested dimensions (up to 100D), the inherent "curse of dimensionality" remains a significant challenge for all metaheuristics. The performance of Helios-AS, like other algorithms, still degrades as problem dimensionality increases (Fig. 2b). Achieving near-optimal solutions in *extremely* high-dimensional spaces (e.g., beyond 100 dimensions) remains an area where current adaptive mechanisms may require further refinement.

5. Current Scope: The current implementation of Helios-MP is designed for single-GPU execution. Scaling to multi-GPU and multi-node environments presents a significant engineering challenge involving inter-GPU communication and load balancing.

These limitations define clear avenues for future research, as outlined in Section VI.

VI. Conclusion and Future Work

The inherent conflict between the complex, state-based logic of sophisticated serial optimizers and the high-throughput paradigm of modern GPUs represents a fundamental barrier to progress in large-scale scientific computing. In this work, we addressed this challenge directly through the principled co-design of the Helios system. We have presented two distinct but philosophically linked implementations: Helios-AS, a novel, landscape-aware serial algorithm proven to be competitive with state-of-the-art methods, and Helios-MP, its massively parallel counterpart which successfully distills the intent of the serial logic into a purely GPU-native workflow. Our comprehensive experimental analysis demonstrates that this co-design approach is highly effective, yielding speedups of up to $15\times$ over an already-optimized baseline without sacrificing the high solution quality required for scientific applications.

Our findings are consistent with the No Free Lunch (NFL) theorems for optimization [15], which posit that an algorithm's performance is derived from its specialization for a particular class of problems. The success of Helios-AS validates its architecture as one such effective specialization for rugged landscapes, providing a strong foundation for our parallelization efforts.

The broader contribution of this paper is a successful blueprint for translating algorithmic intelligence across disparate computational architectures. Our methodology—of identifying core serial mechanisms and inventing novel, parallel-friendly analogues like dynamic regimes and stagnation shockwaves—provides a valuable case study for other researchers seeking to accelerate complex, adaptive algorithms on many-core hardware. We have shown that the path to performance is not always direct parallelization, but a thoughtful reimagining of the algorithm itself to align with the strengths of the target architecture.

The Helios system provides a robust foundation for numerous exciting avenues of future research. Our immediate focus will be on three areas:

- **Scaling to Distributed Systems:** We plan to extend Helios-MP to multi-GPU and multi-node environments. This will involve designing hierarchical population structures and investigating efficient communication strategies using libraries like NCCL to tackle optimization problems at the scale of modern supercomputers.
- **Expanding Scientific Applications:** We will apply Helios to a wider range of challenging domains, including hyperparameter optimization in deep learning and molecular docking simulations in computational drug discovery, where the efficiency of Helios-AS and the speed of Helios-MP can both provide significant value.
- **Algorithmic Enhancement:** Finally, we aim to improve the high-dimensional scalability of the core algorithm by incorporating principles from model-based methods like CMA-ES, further advancing the state-of-the-art.

Ultimately, Helios is not just a new optimizer, but a step towards a future where the most intelligent algorithms can fully exploit the power of the most advanced high-performance hardware.

ACKNOWLEDGMENT

The authors acknowledge Pradhi AI Solutions Priavte Limited for providing computational resources, tools, and research support that facilitated this work. The authors also acknowledge the use of AI-assisted tools for language refinement and stylistic improvement throughout the manuscript, constituting a disclosure of AI assistance as per IEEE policy.

APPENDIX A
EXPERIMENTAL CONFIGURATION DETAILS

This appendix provides comprehensive details of the experimental configuration, hyperparameter settings, and complete benchmark results for the Helios system. The hyperparameter configurations presented here were systematically tuned through preliminary experiments to achieve optimal performance across diverse optimization landscapes.

A. Hyperparameter Settings

Tables III and IV present the complete hyperparameter configurations used for Helios-AS and Helios-MP implementations throughout all experiments. These parameters were carefully selected based on extensive preliminary testing across multiple benchmark functions and represent the optimal balance between exploration and exploitation capabilities.

B. Comprehensive Benchmark Results

Table V presents the complete experimental results across all tested configurations. The table shows mean fitness values achieved over 5 independent runs for each algorithm-function-dimension combination. Lower values indicate better performance, with the best result for each test case highlighted in bold. These results demonstrate the competitive performance of Helios-AS across diverse optimization landscapes and dimensionalities.

The experimental results demonstrate several key findings: (1) Helios-AS achieves the best overall performance with the lowest mean fitness (8.21), (2) CMA-ES performs competitively, particularly excelling on the Rosenbrock function, (3) performance degradation with increasing dimensionality is most pronounced for PSO, while Helios-AS maintains relatively stable performance scaling, and (4) the multimodal functions (Rastrigin, Ackley, Griewank) clearly favor the landscape-aware mechanisms of Helios-AS over traditional approaches.

TABLE III: Helios-AS hyperparameter configuration used in all experiments.

Component	Parameter	Value
Population	Explorer Population	50
	Refiner Population	50
	Total Population	100
	Min Population Ratio	0.25
Iterations	Maximum Iterations	2,500
	Stagnation Threshold	800
	Population Decay Exponent	2.0
Search Operators	Lévy Beta Parameter	1.5379
	DE Scaling Factor (F)	0.3791
	DE Crossover Rate (CR)	0.5012
	Lévy Probability	0.3114
Local Search	L-BFGS-B Trigger	100 iter stagnation
	Explorer Memory Size	10
	Neighborhood Radius	$0.6322 \times$ scale
Surrogate Model	k-NN Neighbors	5
	History Buffer Size	2,000
	Multimodal CV Threshold	0.5
Escape Mechanisms	DE Probability	0.6860
	Diversity Restart Threshold	0.04

TABLE IV: Helios-MP hyperparameter configuration used in all experiments.

Component	Parameter	Value
Population	Total Population	50,000
	Elite Fraction	2.5%
Iterations	Maximum Iterations	1,000
	BB Search Frequency	Every 15 iterations
Search Operators	Lévy Beta Parameter	1.5
	DE Scaling Factor (F)	0.5
	DE Crossover Rate (CR)	0.9
Parallel Mechanisms	Dynamic Regimes	3 modes
	Stagnation Window	50 iterations
	Shockwave Magnitude	$0.1 \times$ bounds

TABLE V: Comprehensive benchmark results showing mean fitness values across all tested configurations. Lower values indicate better performance. Best results for each test case are highlighted in bold.

Test Configuration	Helios-AS	CMA-ES	COBYLA	DE	PSO
Dimension-wise Performance (Averaged across all functions)					
10D Problems	**0.00e+00**	3.69e+00	1.94e+01	1.24e-01	8.81e+00
30D Problems	**3.42e+00**	9.97e+00	3.45e+01	2.15e+01	6.32e+02
50D Problems	**9.72e+00**	1.96e+01	6.11e+01	7.53e+01	8.53e+03
100D Problems	**1.97e+01**	4.83e+01	1.44e+02	2.26e+02	8.87e+04
Function-wise Performance (Averaged across all dimensions)					
Sphere Function	2.40e-05	**0.00e+00**	**0.00e+00**	2.63e-03	7.66e+02
Rastrigin Function	**8.93e-04**	1.01e+02	2.65e+02	3.25e+02	1.87e+02
Rosenbrock Function	4.11e+01	**6.98e-01**	3.22e+01	6.85e+01	1.21e+05
Ackley Function	**3.20e-03**	2.89e-02	1.94e+01	9.45e+00	7.04e+00
Griewank Function	**0.00e+00**	3.39e-03	6.87e+00	1.13e-02	9.53e+00
Overall Performance Summary					
Overall Mean Fitness	**8.21e+00**	2.04e+01	6.47e+01	8.06e+01	2.45e+04

REFERENCES

[1] D. J. Wales and J. P. K. Doye, "Global optimization by basin-hopping and the lowest energy structures of lennard-jones clusters containing up to 110 atoms," *Journal of Physical Chemistry A*, vol. 101, no. 28, pp. 5111–5116, 1997.

[2] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.

[3] S. Basterrech, P. Krömer, and V. Snášel, "GPU parallelization strategies for metaheuristics: A survey," *Artificial Intelligence Review*, vol. 54, pp. 4931–4970, 2021.

[4] X.-S. Yang, S. Deb, and A. H. Gandomi, "Survey of lévy flight-based metaheuristics for optimization," *Mathematics*, vol. 10, no. 15, p. 2785, 2022.

[5] J. Brest, S. Greiner, B. Bošković, M. Mernik, and V. Žumer, "Self-adapting control parameters in differential evolution: A comparative study on numerical benchmark problems," *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 6, pp. 646–657, 2006.

[6] K. Li, X. Fu, F. Wang, and H. Jalil, "A dynamic population reduction differential evolution algorithm combining linear and nonlinear strategy piecewise functions," *Concurrency and Computation: Practice and Experience*, vol. 34, no. 6, p. e6773, 2022.

[7] H. Tong, C. Huang, L. L. Minku, and X. Yao, "Surrogate models in evolutionary single-objective optimization: A new taxonomy and experimental study," *Information Sciences*, vol. 562, pp. 414–437, 2021.

[14] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, "CuPy: A NumPy-compatible library for NVIDIA GPU calculations," in *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017, available at: http://learningsys.org/nips17/assets/papers/paper_16.pdf.

[8] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, "A limited memory algorithm for bound constrained optimization," *SIAM Journal on Scientific Computing*, vol. 16, no. 5, pp. 1190–1208, 1995.

[9] S. García, J. Prada, P. Toharia, Óscar D. Robles, and L. Pastor, "Exploiting multi-level parallel metaheuristics and heterogeneous computing to address an important multiobjective problem in bioinformatics," *Future Generation Computer Systems*, vol. 125, pp. 738–749, 2021.

[10] B. Huang, R. Cheng, Z. Li, Y. Jin, and K. C. Tan, "EvoX: A distributed GPU-accelerated framework for scalable evolutionary computation," *IEEE Transactions on Evolutionary Computation*, 2024, published version of arXiv:2301.12457.

[11] T. Götze and H. Meyerhenke, "Scalable shared-memory hypergraph partitioning," *Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*, pp. 16–30, 2021.

[12] H. Meyerhenke, P. Sanders, and C. Schulz, "Parallel graph partitioning for complex networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 9, pp. 2625–2638, 2017.

[13] P. Hijma, S. Heldens, A. Sclocco, B. van Werkhoven, and H. E. Bal, "Optimization techniques for GPU programming," *ACM Computing Surveys*, vol. 56, no. 6, pp. 1–81, 2023.

[15] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.